

бюджетное профессиональное образовательное учреждение
Вологодской области
«Череповецкий лесомеханический техникум им. В.П. Чкалова»

Специальность 09.02.07 Информационные системы и программирование

ОТЧЁТ ПО ПРЕДДИПЛОМНОЙ ПРАКТИКЕ

Выполнил студент: 4 курса, группы ИС-41

фио
подпись: _____

Место практики: _____
наименование юридического лица, фио ИП

Период прохождения:
с "" 2026 г.
по " 2026 г.

Руководитель практики от предприятия:
должность _____

подпись _____ / _____ /

Руководитель практики от
техникума: Леонова Анна
Васильевна
оценка: _____
" ____ " _____ 2026 г.

МП

г. Череповец
2026

СОДЕРЖАНИЕ

ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ	3
1.1 Организация производства	3
1.2 Структура и функции подразделения базы практики	4
РЕАЛИЗАЦИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ.....	6
2.1 Сравнительный анализ аналогов проектируемой системы	6
2.2 Выбор технологии, среды и языка программирования.....	8
2.3 Анализ процесса обработки информации, выбор структур данных	9
2.4 Проектирование структур данных	9
2.5 Проектирование интерфейса пользователя.....	11
2.6 Реализация выбранных методик и паттернов программирования	13
МЕРОПРИЯТИЯ ПО ОХРАНЕ ТРУДА И ТЕХНИКЕ БЕЗОПАСНОСТИ	16
3.1 Общие требования безопасности	16
3.2 Требования безопасности перед началом работы	16
3.3 Требования безопасности во время работы	16
3.4 Требования безопасности в аварийных ситуациях	17

ОБЩАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ

ООО «Малленом Системс» была создана в 2011 году на базе команды ученых и программистов Санкт-Петербургского политехнического университета Петра Великого. Сегодня в компании более 100 сотрудников. Глубокие компетенции в сфере машинного зрения и большой опыт успешной реализации проектов на промышленных предприятиях позволяет успешно решать большой спектр задач в различных отраслях. В Центре исследований и разработки интеллектуальных систем ведется работа по созданию новых решений и развитию продуктов компании.

1.1 Организация производства

Разработка информационных систем являются основной деятельностью ООО «Малленом Системс». ИТ-инфраструктура компании построена на современных технологических решениях:

- Платформы и языки разработки: Используется платформа .NET (C#) и среда разработки [ASP.NET](#) Core для создания высокопроизводительных веб-приложений и микросервисов.
- Работа с данными: Применяются реляционные СУБД PostgreSQL и MS SQL Server. Взаимодействие с базами данных реализуется через технологию Entity Framework Core (ORM). Общение между сервисами происходит по REST API.
- Развертывание и контроль версий: Для обеспечения надежной работы сервисов применяются облачные решения (Yandex Cloud) и системы контейнеризации (Docker, Kubernetes). Управление исходным кодом ведется с использованием систем контроля версий Git и GitLab.

1.2 Структура и функции подразделения базы практики

ИТ-инфраструктура является фундаментом для деятельности всей компании. Функции ИТ-подразделения включают:

- Администрирование инфраструктуры: Использование специализированного ПО для администрирования сетей и оборудования, мониторинга состояния серверов (Zabbix, Grafana).
- Управление ресурсами: Работа с системами управления контентом для поддержки корпоративного портала и документации.
- Мониторинг приложений: Настройка систем логирования для отслеживания работы приложений с использованием инструмента Serilog и Elastic Stack.
- Нормативный контроль: Соблюдение политик информационной безопасности, регламентов проведения резервного копирования и правил работы с конфиденциальными данными в соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных».

В период с 20 апреля по 15 мая 2026 года в ООО «Малленом Системс» мною была пройдена преддипломная практика с целью выполнения выпускной квалификационной работы на тему «Разработка и внедрение информационной системы для автоматизации управления складскими операциями».

Индивидуальное задание на практику включало:

1. Проведение анализа предметной области складского учета.
2. Изучение и анализ существующих WMS-систем.

3. Проектирование базы данных и архитектуры приложения.
4. Разработка серверной части на [ASP.NET](#) Core.
5. Создание пользовательского интерфейса на HTML/CSS/JavaScript.
6. Реализация механизмов аутентификации и разграничения прав доступа.
7. Тестирование разработанного функционала.

РЕАЛИЗАЦИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

2.1 Сравнительный анализ аналогов проектируемой системы

В рамках выполнения индивидуального задания был проведен анализ существующих программных решений для автоматизации складского учета (WMS). Основными критериями оценки выступали: поддержка адресного хранения, управление комплектацией, стоимость внедрения и простота использования.

В качестве аналогов были рассмотрены:

1. 1С:Управление торговлей (модуль «Склад»):

Преимущества: Широкая распространенность, низкая стоимость, наличие квалифицированных специалистов по внедрению.

Недостатки: Ограниченная поддержка адресного хранения, отсутствие развитых механизмов управления комплектацией заказов и работы с терминалами сбора данных (ТСД).

2. МойСклад:

Преимущества: Облачная модель (доступ из любой точки), простота использования, отсутствие затрат на серверное оборудование.

Недостатки: Ограниченная гибкость настройки под специфику бизнеса, зависимость от качества интернет-соединения, ежемесячная абонентская плата.

3. Solvo.WMS:

Преимущества: Профессиональная система для управления складом любой сложности, широчайший функционал (включая управление персоналом и 3D-визуализацию), масштабируемость.

Недостатки: Высокая стоимость лицензий и внедрения, сложность настройки и длительный период внедрения (от полугода).

Таблица 3 — Сравнительный анализ существующих WMS-систем

Критерий	1С:УТ	МойСклад	Solvo.WMS
Адресное хранение	Ограничено	Да	Да
Управление комплектацией	Нет	Базово	Да
Работа с ТСД	Ограничено	Да	Да
Инвентаризация	Да	Да	Да
Сложность внедрения	Низкая	Низкая	Высокая
Стоимость внедрения	Низкая	Средняя	Высокая
Гибкость настройки	Средняя	Средняя	Высокая
Тип развертывания	Локальный	Облачный	Локальный

Проведенный анализ показал, что существующие решения либо слишком дороги и сложны для среднего бизнеса, либо не предоставляют необходимого набора функций для эффективного управления. Это обосновало

необходимость разработки собственной системы, которая сочетает ключевой функционал WMS с доступной стоимостью и простотой использования.

2.2 Выбор технологии, среды и языка программирования

Учитывая специфику задачи (необходимость создания кроссплатформенного веб-приложения с удобным интерфейсом и надежной серверной частью), был выбран следующий технологический стек:

- Язык программирования: C#. Выбор обусловлен высокой производительностью, строгой типизацией, надежностью и мощной экосистемой платформы .NET.
- Серверный фреймворк: [ASP.NET](#) Core 8.0. Обеспечивает высокую производительность, встроенные механизмы Dependency Injection, поддержку REST API и кроссплатформенность.
- Клиентский фреймворк: Чистый HTML/CSS/JavaScript. Выбор обусловлен необходимостью создания легковесного, интуитивно понятного интерфейса без изучения дополнительных сложных фреймворков.
- СУБД: MySQL. Выбрана как надежная, бесплатная и широко распространенная реляционная база данных с поддержкой сложных транзакций, что критически важно для точности складского учета.
- ORM: Entity Framework Core. Позволяет работать с базой данных через объекты C#, упрощая разработку и сокращая объем шаблонного кода.
- Среда разработки: Microsoft Visual Studio 2022. Предоставляет удобные инструменты для разработки на C#, отладки и управления проектом.

2.3 Анализ процесса обработки информации, выбор структур данных

Процесс обработки информации в разработанной системе базируется на клиент-серверной архитектуре и разделен на три основных этапа:

1. Ввод данных: Пользователь (администратор, менеджер или кладовщик) через веб-интерфейс инициирует действие: создание товара, оформление приходной накладной, комплектацию заказа и т.д.
2. Обработка на сервере: Контроллер (API) принимает HTTP-запрос, проверяет права доступа и аутентификацию пользователя, после чего вызывает соответствующий метод сервиса (Service). Сервисный слой содержит бизнес-логику: проверка достаточности товара на складе, пересчет остатков, обновление статусов документов.
3. Хранение данных: Entity Framework Core через LINQ-запросы преобразует C#-объекты в SQL-запросы и сохраняет/извлекает данные из базы данных MySQL.

Для хранения и передачи информации выбраны следующие структуры данных:

- В оперативной памяти: Данные передаются в виде объектов C# (DTO). В JavaScript данные передаются в формате JSON.
- В базе данных: Реляционная модель данных, приведенная к третьей нормальной форме (3NF).

2.4 Проектирование структур данных

Проектирование базы данных началось с разработки концептуальной модели, в которой были выделены основные сущности предметной области:

«Пользователь», «Товар», «Складская ячейка», «Приходная накладная», «Расходная накладная».

Концептуальная модель данных (ER-диаграмма) представлена на Рисунке 1.

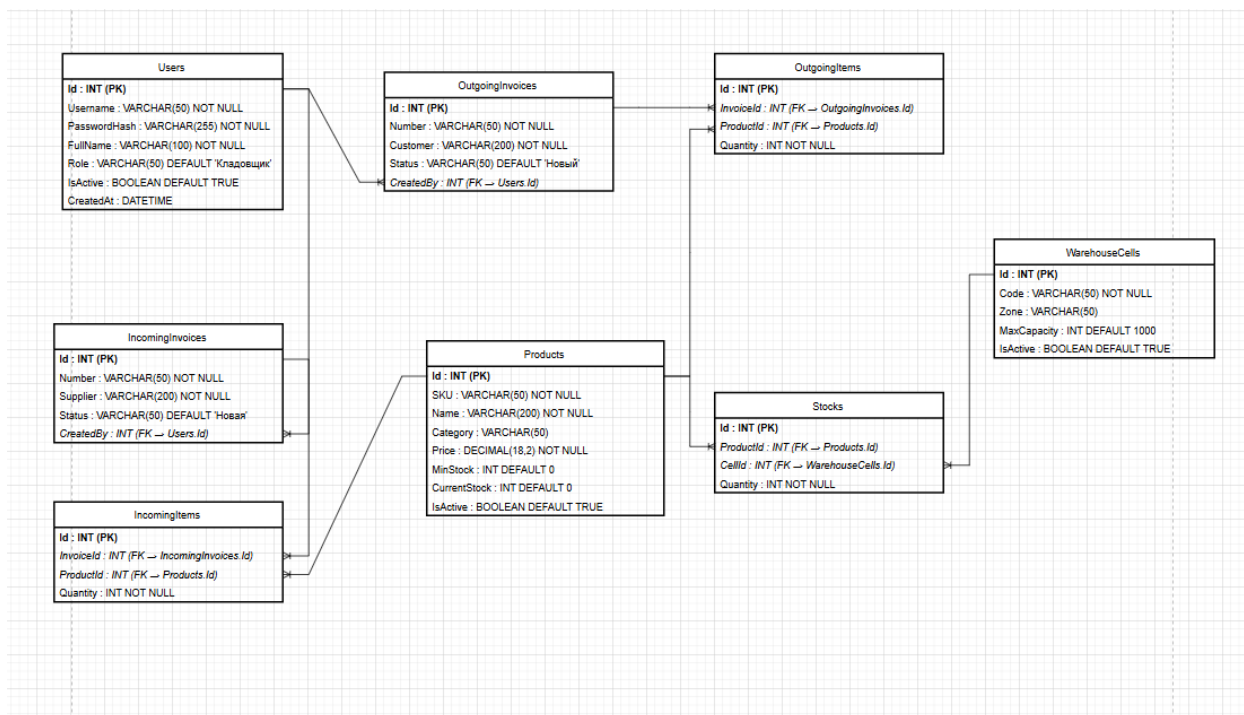


Рисунок 1 - ER-диаграмма

Далее была разработана физическая модель данных, то есть конкретная схема таблиц в СУБД MySQL:

- **Users**: Хранит учетные записи пользователей (Id, Username, PasswordHash, FullName, Role, IsActive...).
- **Products**: Хранит информацию о товарах (Id, SKU, Name, Category, Price, MinStock, CurrentStock...).
- **WarehouseCells**: Хранит информацию о складских ячейках (Id, Code, Zone, MaxCapacity...).
- **IncomingInvoices/IncomingItems**: Хранят приходные накладные и их позиции.

- OutgoingInvoices/OutgoingItems: Хранят расходные накладные и их позиции.

SQL-скрипт создания структуры базы данных представлен на Рисунке 2.

```
{
    migrationBuilder.CreateTable(
        name: "Products",
        columns: table => new
        {
            Id = table.Column<int>(type: "int", nullable: false)
                .Annotation("SqlServer:Identity", "1, 1"),
            SKU = table.Column<string>(type: "nvarchar(50)", maxLength: 50, nullable: false),
            Name = table.Column<string>(type: "nvarchar(200)", nullable: false),
            Category = table.Column<string>(type: "nvarchar(50)", nullable: true),
            Price = table.Column<decimal>(type: "decimal(18,2)", nullable: false),
            MinStock = table.Column<int>(type: "int", nullable: false, defaultValue: 0),
            CurrentStock = table.Column<int>(type: "int", nullable: false, defaultValue: 0),
            IsActive = table.Column<bool>(type: "bit", nullable: false, defaultValue: true)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_Products", x => x.Id);
        });
}
```

Рисунок 2 – Фрагмент кода миграции

Все таблицы нормализованы для устранения избыточности данных.

2.5 Проектирование интерфейса пользователя

Интерфейс проектировался с учетом трех категорий пользователей: администратор, менеджер, кладовщик. Для каждой роли был разработан свой набор экранов и функциональность.

Основные принципы проектирования интерфейса:

- Интуитивная понятность для пользователей с разным уровнем технической подготовки.
- Минималистичный дизайн для снижения зрительной нагрузки (реализована поддержка темной и светлой темы).
- Отображение критически важной информации (остатки, статусы заказов) в реальном времени.

Страница авторизации пользователя представлена на Рисунке 3.

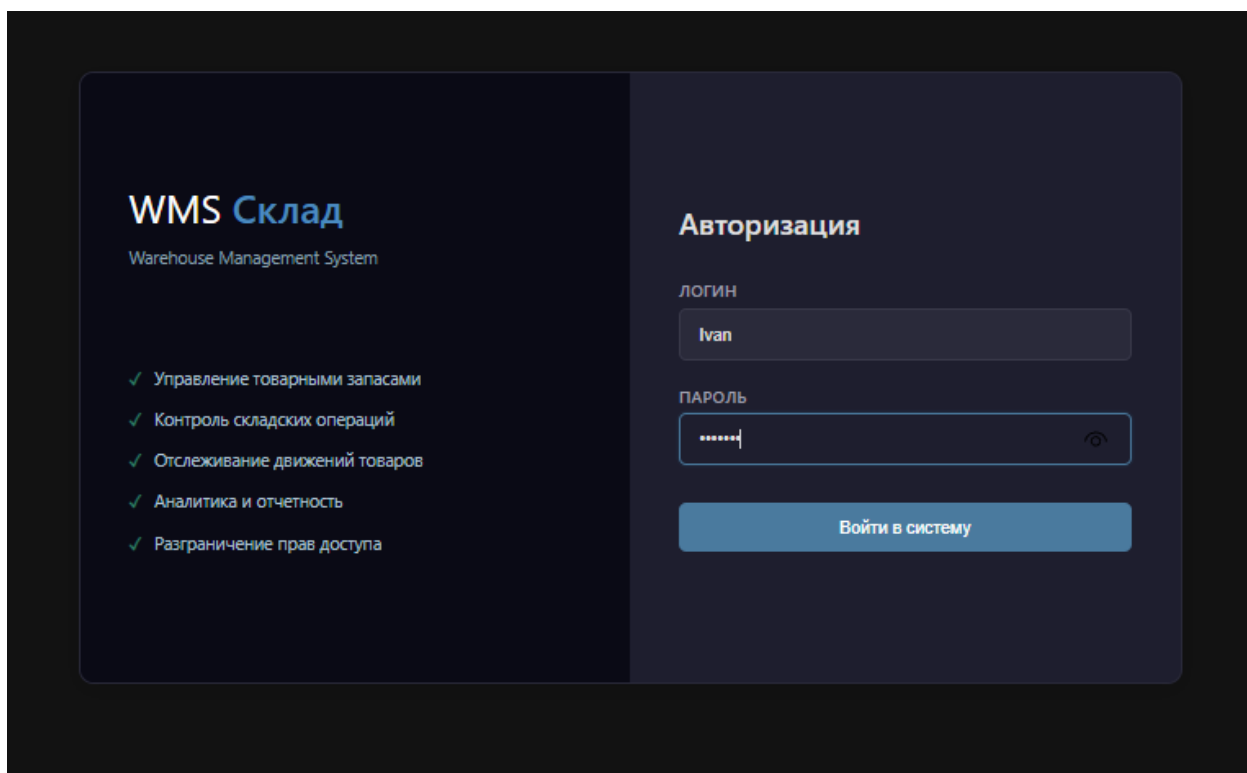


Рисунок 3 - Окно авторизации

Панель администратора представлена на Рисунке 4.

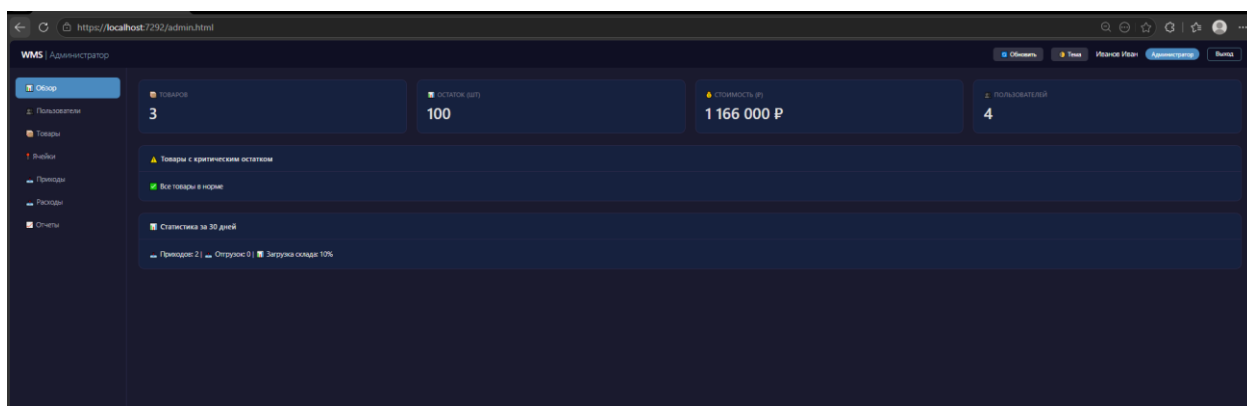


Рисунок 4 - Интерфейс администратора

Рабочие места менеджера и кладовщика выполнены в едином стиле, но с разным набором доступных функций. Интерфейс кладовщика (раздел «Приемка товара») представлен на Рисунке 6.

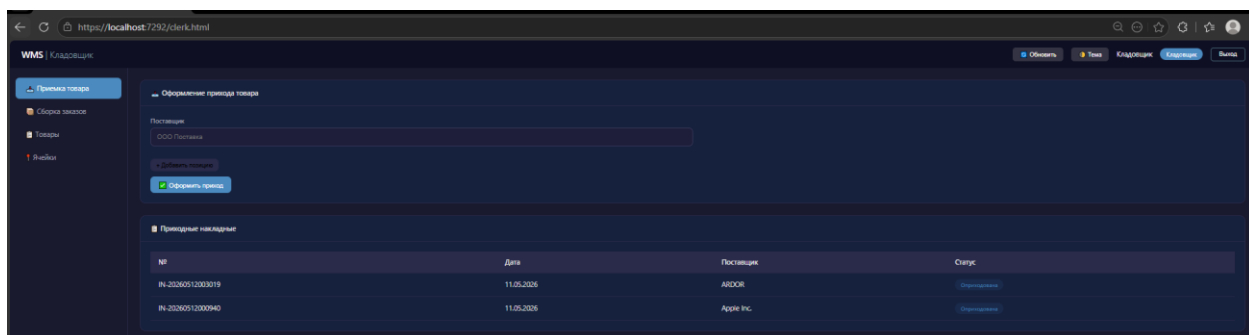


Рисунок 5 - Интерфейс кладовщика

2.6 Реализация выбранных методик и паттернов программирования

Архитектура программного комплекса построена на принципе разделения ответственности и включает три основных уровня: уровень представления (Frontend), уровень бизнес-логики (Backend) и уровень доступа к данным (Repository/Services). Применены следующие паттерны и методики:

1. MVC (Model-View-Controller) на сервере: Хотя контроллеры в проекте выполняют функции API, логически код разделен на модели (сущности БД), представления (фронтенд) и контроллеры (обработка запросов).
2. Репозиторий (Service Layer): Вся бизнес-логика (например, метод ReceiveGoods, обновляющий остатки) вынесена в отдельный класс WarehouseService. Контроллеры обращаются к методам этого сервиса, что обеспечивает переиспользование кода и упрощает тестирование.

```

Ссылка 1
public async Task<IncomingInvoice?> ReceiveGoods(IncomingInvoice invoice, List<IncomingItem> items)
{
    using var transaction = await _context.Database.BeginTransactionAsync();
    try
    {
        invoice.CreatedAt = DateTime.UtcNow;
        invoice.Status = "Оприходована";
        await _context.IncomingInvoices.AddAsync(invoice);
        await _context.SaveChangesAsync();

        foreach (var item in items)
        {
            item.InvoiceId = invoice.Id;
            await _context.IncomingItems.AddAsync(item);

            var product = await _context.Products.FindAsync(item.ProductId);
            if (product != null)
            {
                product.CurrentStock += item.Quantity;
                _context.Entry(product).State = EntityState.Modified;
            }
        }

        await _context.SaveChangesAsync();
        await transaction.CommitAsync();
        return invoice;
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync();
        Console.WriteLine($"Ошибка: {ex.Message}");
        throw;
    }
}

```

Рисунок 6 - Метод ReceiveGoods

3. Ролевая модель (RBAC): Реализована система разграничения прав доступа на основе ролей («Администратор», «Менеджер», «Кладовщик»). В зависимости от роли пользователю показываются разные разделы интерфейса.
4. DTO (Data Transfer Object): Для передачи данных между сервером и клиентом используются отдельные классы (например, LoginRequest, ReceiveRequest), которые содержат только необходимые поля, что снижает нагрузку на сеть и повышает безопасность.

Результат выполнения индивидуального задания:

В ходе практики была разработана, запущена и протестирована работающая версия информационной системы для автоматизации управления складскими операциями.

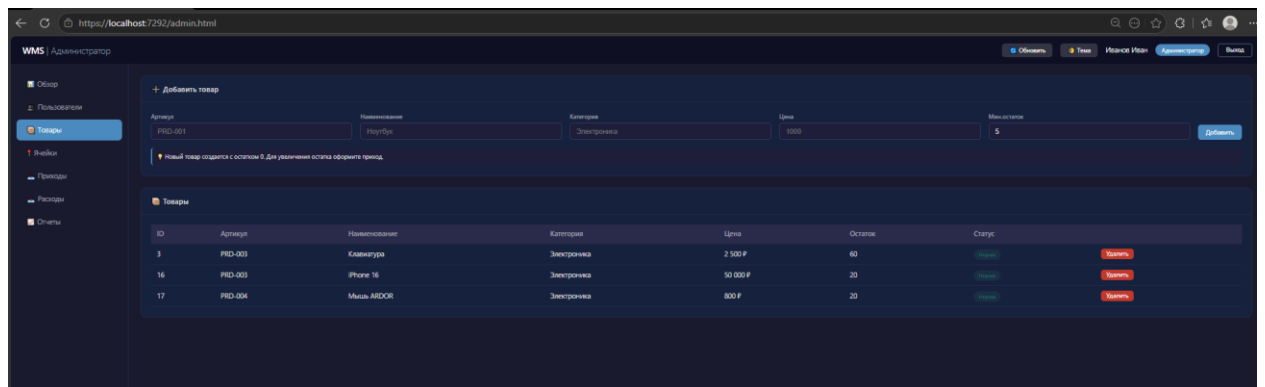


Рисунок 7 - Раздел "Товары"

МЕРОПРИЯТИЯ ПО ОХРАНЕ ТРУДА И ТЕХНИКЕ БЕЗОПАСНОСТИ

3.1 Общие требования безопасности

- К самостоятельной работе за ПЭВМ допускаются лица не моложе 18 лет, прошедшие вводный инструктаж и инструктаж на рабочем месте, не имеющие медицинских противопоказаний.
- Основными опасными и вредными производственными факторами для ИТ-специалиста являются: электромагнитное излучение, повышенная нагрузка на зрение, статическая рабочая поза, повышенный уровень шума.
- Помещение для работы должно быть оборудовано защитным заземлением и системой пожарной безопасности.

3.2 Требования безопасности перед началом работы

- Произвести визуальный осмотр рабочего места: проверить исправность розеток, вилок, кабелей питания ПК и монитора, целостность корпусов.
- Отрегулировать положение экрана монитора, кресла и подставки для ног для принятия удобной рабочей позы.
- Работа с информационными ресурсами в организации строго регламентирована федеральным законодательством (ФЗ-149 «Об информации...», ФЗ-152 «О персональных данных») и локальными актами. Перед началом работы необходимо убедиться в соблюдении внутренних политик безопасности.

3.3 Требования безопасности во время работы

- Для снижения зрительного и статического напряжения необходимо делать перерывы по 10-15 минут каждый час работы за монитором, выполнять упражнения для глаз и разминку.
- Расстояние от глаз до экрана монитора должно составлять 60-70 см.
- Во время работы необходимо соблюдать правила цифровой гигиены: не оставлять сессии без пароля, не передавать свои учетные данные третьим лицам, соблюдать инструкции по работе с конфиденциальными данными предприятия.

3.4 Требования безопасности в аварийных ситуациях

- В случае появления запаха гари, дыма, искрения от техники или срабатывания пожарной сигнализации — немедленно отключить оборудование от электросети и покинуть помещение.
- Сообщить о неполадках руководителю.
- В случае программных сбоев опираться на техническую документацию и регламенты проведения резервного копирования для восстановления работоспособности систем без потери данных.